



MCPCon + AGNTCon Europe 2026



Credent**Agent** Project

Agents Can Pay. Can They Prove It?

The consent layer for AI agents

Diego Zuluaga

Creator & Maintainer, CredentAgent

Head of Solution Architecture, Futurewei

AAIF Ambassador

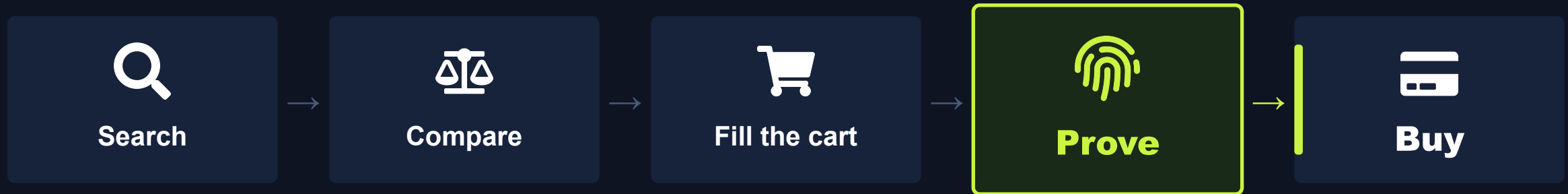


Connect on LinkedIn



Your agent just hit identity

An agent can search, compare and fill a cart on its own. To buy, it has to show some proofs



The agent handles these alone.

Proof first, then the purchase.



Over 21?

- Age gate on a restricted item
- Returns one field only



A real member?

- A discount or entitlement
- Tied to a genuine membership



Approved by the person?

- A payment the human authorised
- On their own device

WHAT IS ALREADY SOLVED

A model cannot reason its way to any of these. So where does the proof come from?
The proof is already in your pocket, on your device.



What is a digital credential?

Something official about you that your phone can prove.

IN THE WALLET APP

Driving licence

Name

Date of birth

Address

Licence number

Over 21

Over 21: yes
issuer signed

One question asked. One signed answer returned.



Issuer signed

The issuer is the authority that vouches for you. Its signature can be checked offline.

a government agency · a bank · an airline · a university



Device bound

The key lives on the secure chip. It cannot be copied to another phone



Selectively disclosed

Ask for age, it returns over 21. Nothing else leaves.

YOU ALREADY HAVE ONE

Yivi has carried a Dutch driving licence since January — scan the chip, and it presents over OpenID4VP. The state's own NL Wallet is a public repo: github.com/MinBZK/nl-wallet

27

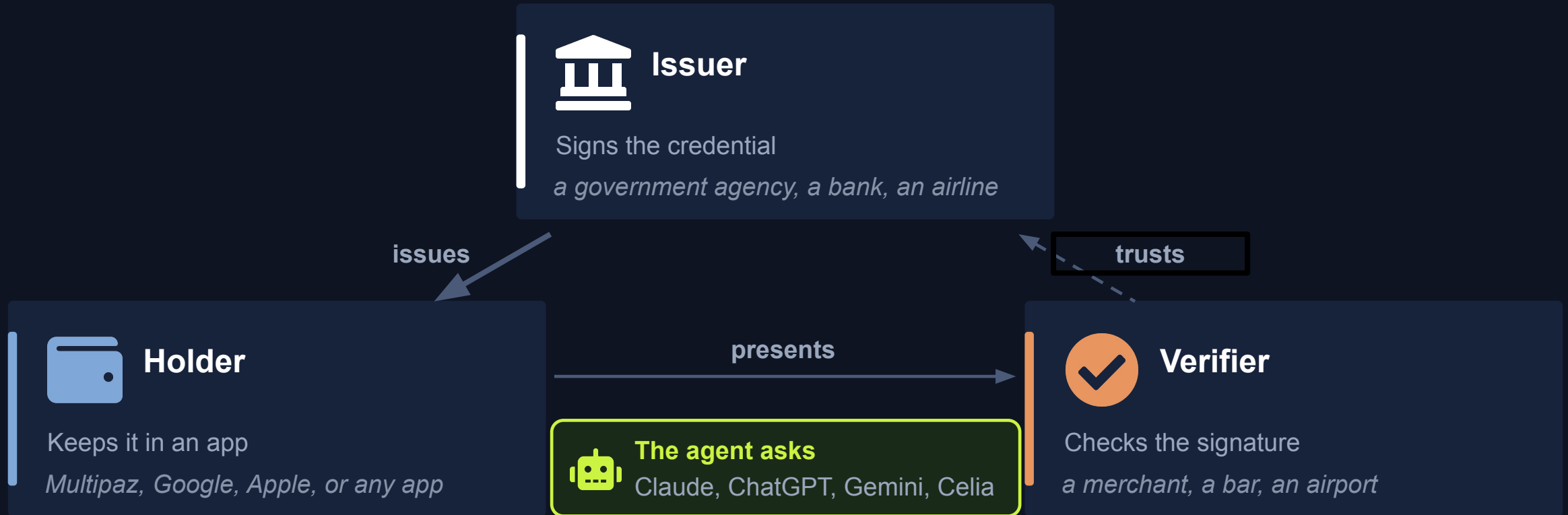
MEMBER STATES MUST OFFER A WALLET

Italy, France and Spain already do — the deadline is December 2026.



The triangle of trust

Three roles, one credential. The agent asks, the wallet answers, the verifier checks the signature.



THE POINT

The verifier never has to know the holder, only the issuer · One integration reaches every wallet on every platform · The agent carries the request and never holds the data



What is Multipaz?

This is what an agent builds against. An open source SDK for digital credentials. It can be used to ask for proof before a purchase completes. CredentAgent is built on top of it.



The SDK

- Kotlin Multiplatform
- Android, iOS, JVM, JS, WASM
- Wallet, verifier and issuer in one library
- Keys to digital credential stays in the phone's security chip



The Wallet

A full wallet app on Android and iOS. APKs at apps.multipaz.org



Reference apps

Identity Reader, web verifier, issuer portal, Utopia test universe



Standards built in

ISO mdoc, SD-JWT VC, ISO 18013-5 and 7, OpenID4VP, W3C DC API



developer.multipaz.org — Read the official documentation and developer guides



Started at Google, built in the open

Eight years from an Android platform API to the credential layer for agents.



THE STARTING POINT

You are holding a hardware-isolated secure element right now

The credentials are already in your wallet. Your agent asks — your wallet presents, sharing only what's asked.

IN PRODUCTION TODAY — ON STANDARDS, WORLDWIDE



Mobile driving licence

ISO 18013-5 mdoc



Digital payment credentials

Tap, online, in-app



Age verification

Selective disclosure, SD-JWT



Transit & boarding passes

Used millions of times a day

Billions

of these credentials are presented and verified every day — in production, on standards, worldwide.

AND THE SAME RAILS CARRY — WHAT YOUR AGENT CAN PROVE NEXT



Identity



Memberships



Prescriptions



Education



Employment



Travel



Health cover



Access

*This is not a prototype. It is **deployed infrastructure**.*



Open standards did the hard part

We stand on the shoulders of giants: the standards bodies did the hard part, the wallet implements it, and CredentAgent only parses and verifies.

Credential formats

ISO 18013-5/7 mdoc

SD-JWT

ISO and IETF defined the formats. The wallet holds them; we parse and verify — we define nothing new.

Request & presentation

OpenID4VP

DCQL

W3C Digital Credentials API

OpenID & W3C defined the ask. DCQL expresses the query; the DC API carries it natively.

Device binding

WebAuthn / FIDO2

FIDO PXP

Secure element

FIDO & W3C bound the device. Origin-bound; cross-device when the desktop asks and the phone answers.

Agentic commerce & settlement

MCP

UCP

ACP

AP2

DPC

x402

The one layer still being written. — MCP moves the agent, ACP/UCP the cart, AP2 authorizes, x402 settles.

MULTIPAZ implements these open standards — donated by Google, improved in the open (Kotlin Multiplatform, one codebase for iOS + Android).

The giants wrote the specs; the wallet stands on them, and so do we.

THE QUESTION



Everyone's racing to invent new identity for AI agents — when it's already in your pocket, in hardware, on open standards.

So when does the agent developer community start building?

Now

Our first release — experimental, and yours to build on.

IN THE NEXT FEW MINUTES

- 1 The flows, working, on real agents
- 2 How the framework fits together
- 3 A two-minute quickstart you can run today



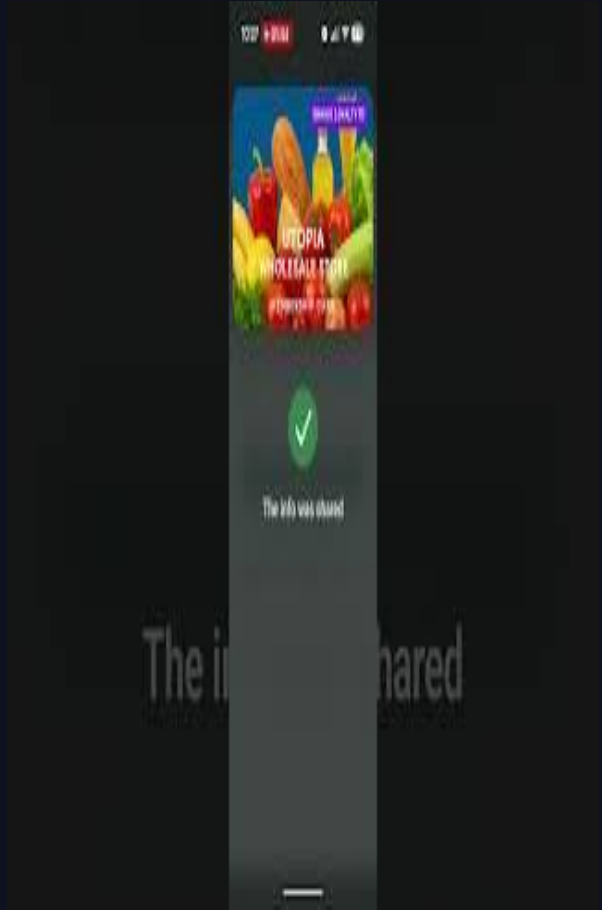
CredentAgent

DOES IT ACTUALLY WORK?

DEMO



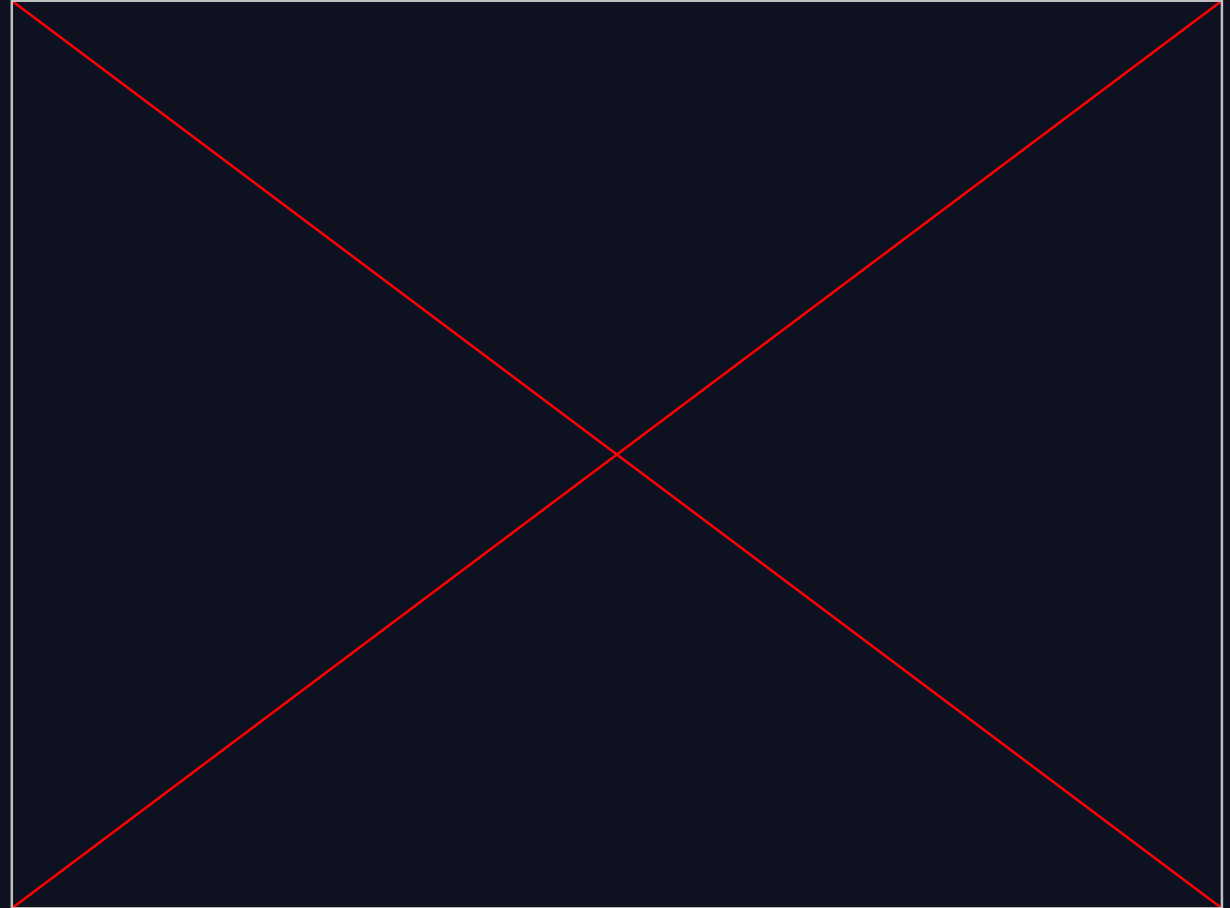
Claude mobile app



ChatGPT mobile app



Goose Desktop



Real agents

Claude, ChatGPT, Goose — not a mock app



Over MCP

Remote connector, no code to try it



Real device

Wallet ceremony on the phone



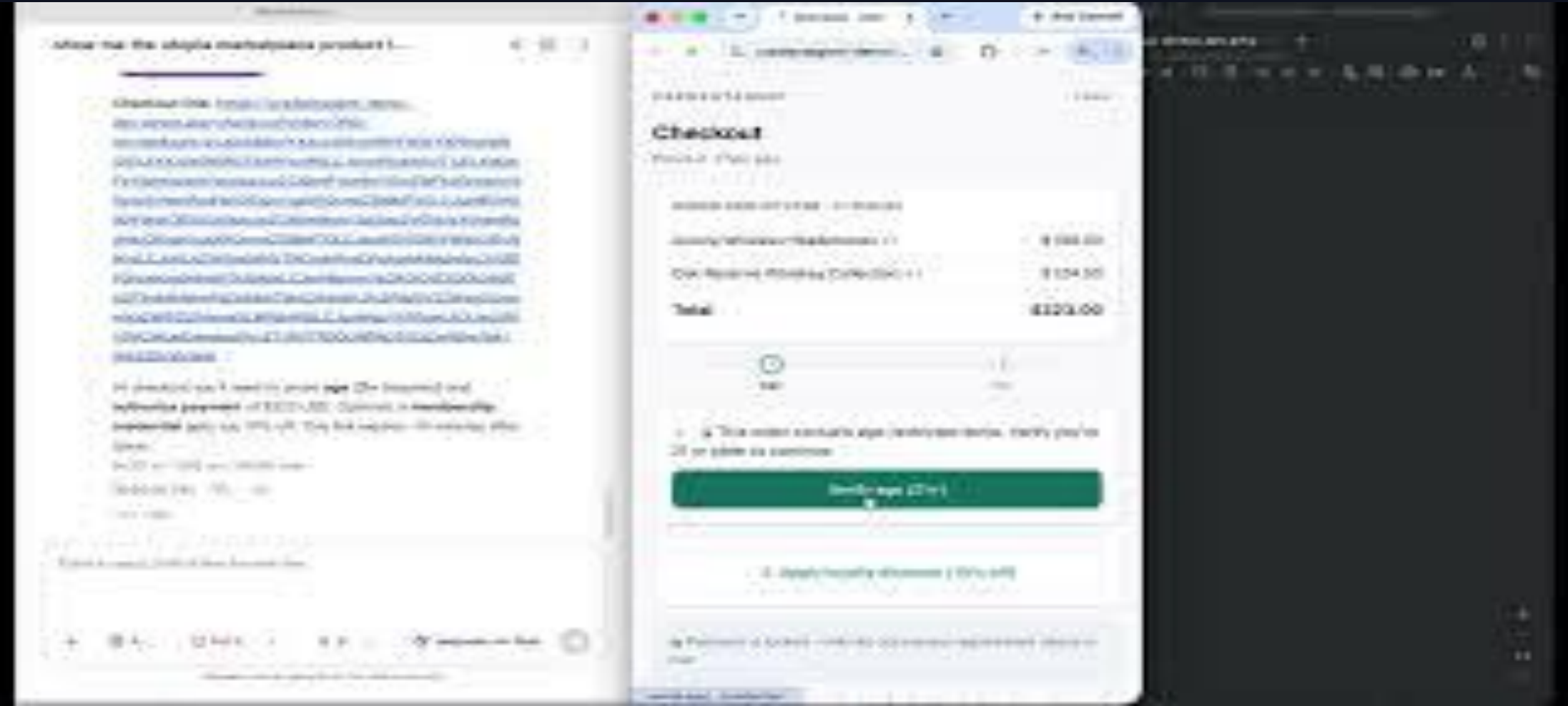
Server-side

Enforced on the completion path

DOES IT ACTUALLY WORK?

DEMO

● WorkSwarm is next. It draws screens with A2UI rather than MCP Apps.



DEMO

x402 Payment

An agent pays for a resource — and the money lands on a real, public ledger.

WATCH FOR

① Consent gate passes

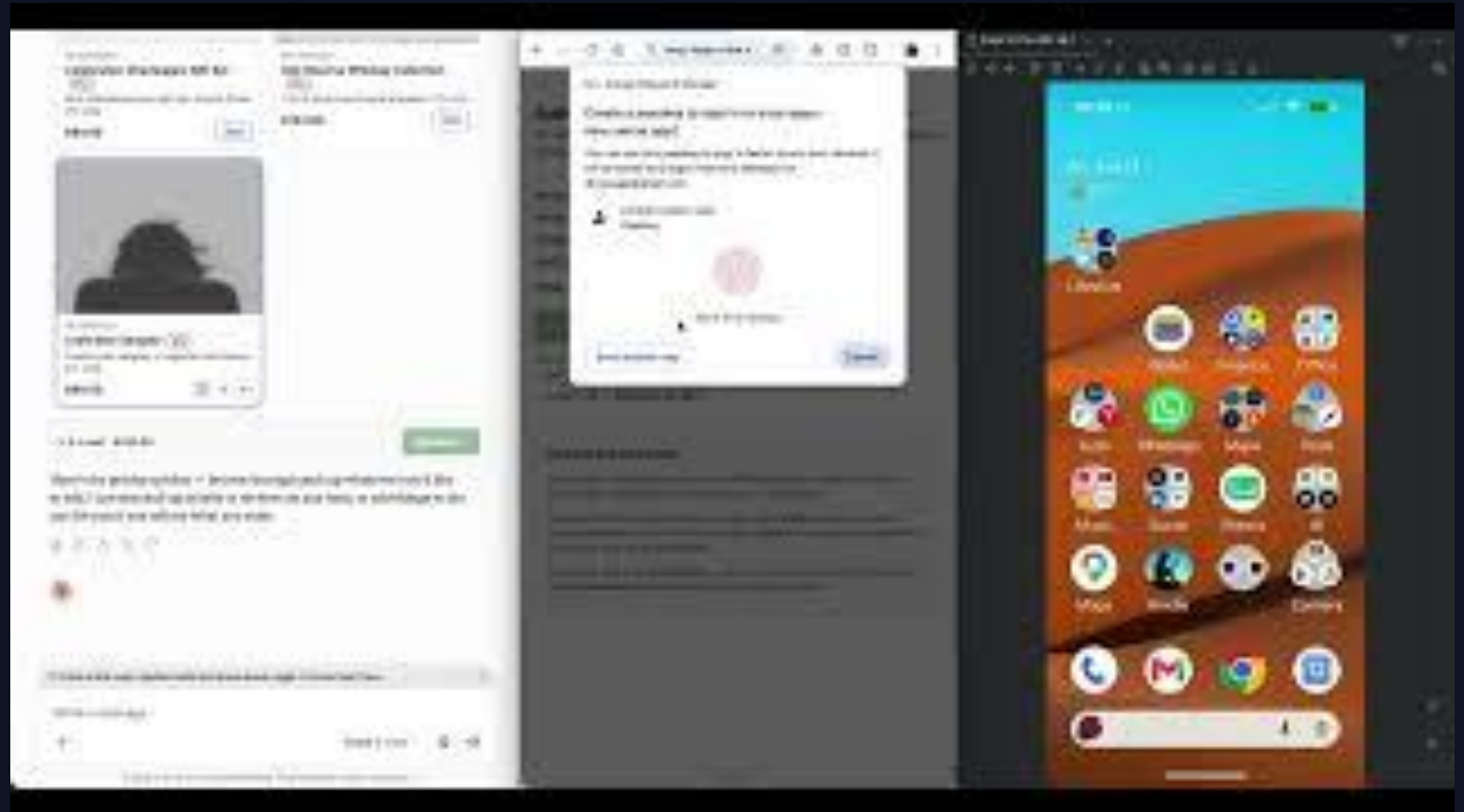
agent presents a credential

② x402 settles

402 → signed payment → 200

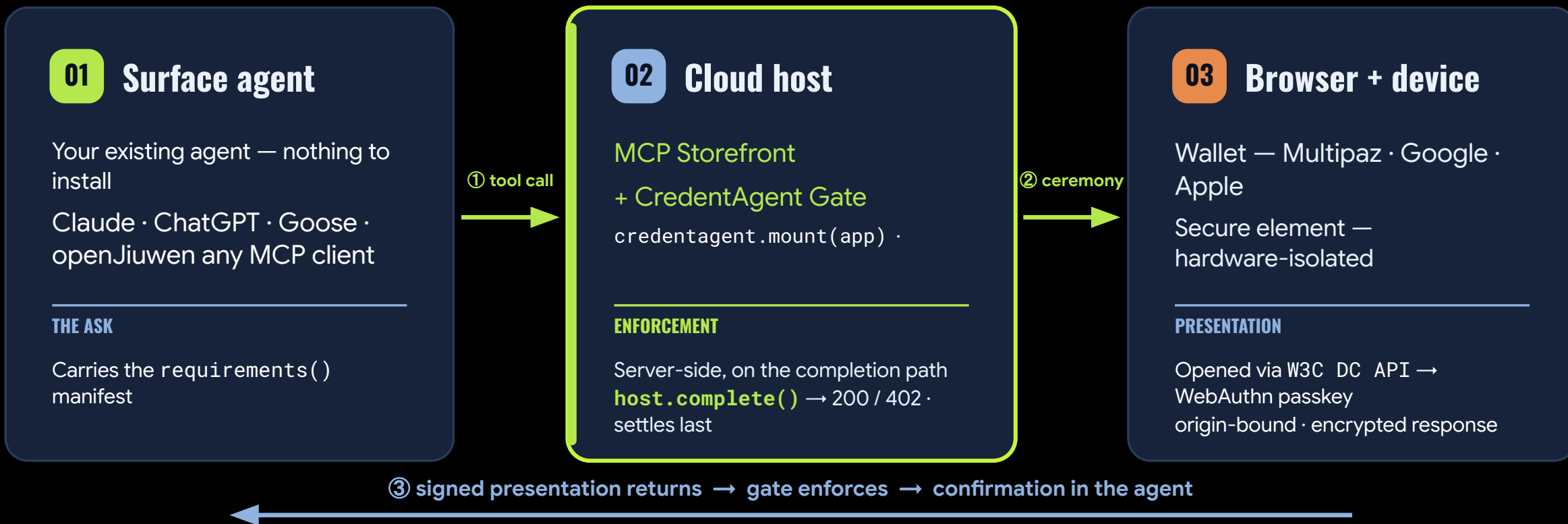
③ It's on-chain

confirms on HashScan in ~3s





Three places the work happens





Two packages, one seam

The Gate is the credential engine. **The Storefront is that engine, already hosted.**

THE PRIMITIVE

The Gate

`@openmobilehub/credentagent-gate`

The credential engine.
Wallet-ceremoany rails + a typed policy → a serializable requires manifest.

Host-agnostic — mounts on any host

THE SEAM

`.mount(app)`

one line = the whole integration

BATTERIES-INCLUDED

✓ Gate inside

The Storefront

`@openmobilehub/credentagent-storefront`

A runnable MCP shop — catalog, MCP tools, and a checkout widget. The Gate is already mounted.

...or bring your own host — same Gate, your server

Both Apache-2.0 · ESM · shipping their own types · github.com/openmobilehub/credentagent · v0.4.0



Stand up a credential-gated store

createStorefront() is a full agentic shopping server — an MCP agent browses, carts, and checks out.

```
$ npm i @openmobilehub/credential-gate @openmobilehub/credential-storefront
```

```
const store = createStorefront();
const credentialagent = new CredentialAgent();
credentialagent.mount(store.app); // ceremony rails

store.gate((order) =>
  credentialagent.requirements(order, [
    required(age.over(21)),
    optional(membership.discount(10)),
    required(payment.in("usd")),
  ]));

await store.listen(3005); // → /mcp, ready for an agent
```

One command

createStorefront() → an MCP server, tools, widget, and checkout.

Policy, not plumbing

age · membership · payment in one ordered array.

Enforced server-side

Every completion path — never the UI.

Settles last

x402 · Hedera, or your own rail.

- An MCP agent (Claude · ChatGPT · Goose) drives the whole checkout over the connector.



Gate any credential you define

age, payment, membership are just built-ins — define and query any doctype the same way.



```
// the built-ins are just pre-defined credentials -  
// query ANY doctype the same way:  
const license = defineCredential({  
  id: "professional_license",  
  request: dcql({ docType: "org.example.license.1",  
    claims: ["license_active"] }),  
  verify: (c) => c.license_active === true,  
  effect: gate(),    // block until proven  
  ui: { label: "Professional license", action: "Verify" },  
});  
  
credentagent.requirements(order, [  
  required(license),    // your credential  
  required(age.over(21)), // + the built-ins  
  required(payment.in("usd")),  
]);
```

● Custom credentials: only **gate()** is wired today.

Define once

id · request · verify · effect · ui. No registration step.

Any doctype

dcql({ docType, claims }) — ISO mdoc or SD-JWT.

Composes

Drops into the same policy array as the built-ins.

Three effects

block · discount · authorize — see next slide.



Three flows, increasing difficulty

① Working

Human present

User is there, proves the credential, action completes. The baseline ceremony.

② Working

Human present at purchase

Agent works ahead; the human arrives only at the consequential moment.

③ In progress

Human not present

Pre-authorized mandate, agent acts alone. Where the hard problems live.



The same flows, rendered in the conversation

Working

Human present

The catalog widget, inline in Claude or ChatGPT. Cart edits sync both ways with the agent.



AUDIO

Aurora Wireless Headphones

Over-ear ANC headphones with 40h battery life.

\$199.00

 1 



BEVERAGES

Oak Reserve Whiskey Collection

21+

Trio of small-batch aged whiskeys. 21+ only.

\$124.00







 3 in cart · \$297.00



Working

Human present at purchase

The consent moment. Approve and Decline both open the ceremony page — never the widget.

CREDENTAGENT

Needs approval

Approve a spending grant

Your agent will spend at Utopia Marketplace while you're away. Here's exactly what you're allowing:

WHAT

Beverages

HOW MUCH

\$250 total · \$90 per purchase

WHO

Your agent spends **unattended** until the budget runs out — or you revoke it.

Approve

Decline

Opens the CredentAgent approval page ↗

delegated-demo · limits enforced server-side · consent is dev-sealed, not wallet-signed

In progress

Human not present

A standing grant: budget depletes as the agent spends. Revocable at any time.

SPENDING GRANT

Active



Aurora Wireless Headphones

Audio · \$199 each

up to 1 per purchase · \$199 max

\$199 of \$398 remaining

up to \$199 per purchase

Spends at Utopia Marketplace while you're away

Revoke

delegated-demo · limits enforced server-side · consent is dev-sealed, not wallet-signed



AP2 chains three mandates. So do we.

① INTENT

SIGNED BY YOU

What you authorized — up front.

running shoes ≤ \$200
by Fri · Nike/Adidas

LOCKS budget cap · expiry · product ·
merchants · which credentials

*Your standing rule. The agent can never
spend past it.*

② CART

SIGNED BY MERCHANT

Exactly what's being bought
— sealed.

Nike Pegasus 41
\$178.00 · final · Tue

LOCKS exact line items · tamper-evident
total · expires in a minute

*The merchant is bound to this cart. No
drift, no ambiguity.*

③ PAYMENT

SIGNED BY AGENT

The deal, closed —
autonomously.

charge Visa ••4242
\$178.00 · one-time

LOCKS one amount · one order · one time

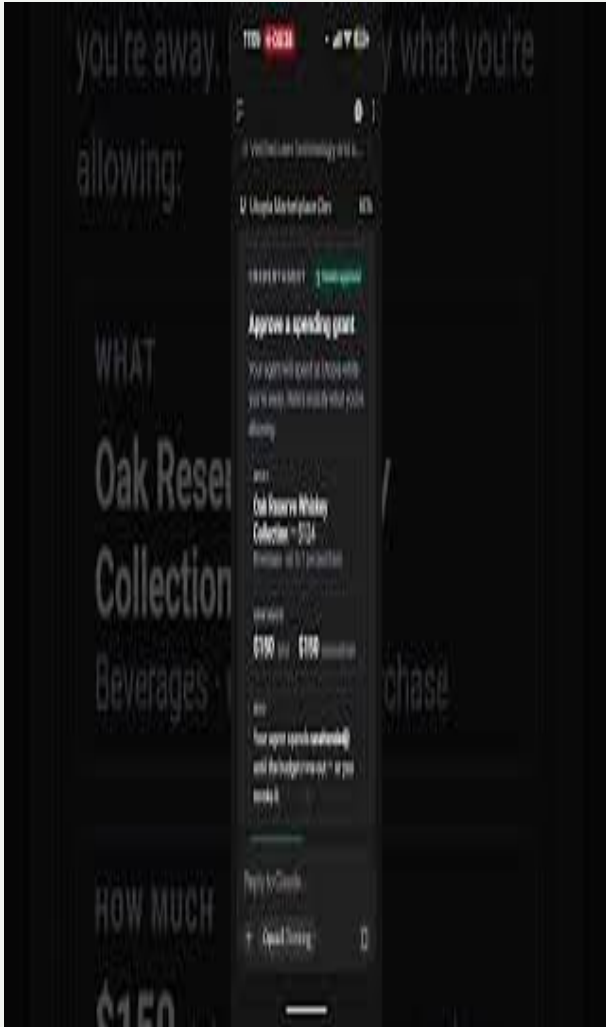
*The agent settles inside your intent.
Hands-free.*

All three must agree before anything settles. `cart.total === re-derived === payment.amount`

Model aligned. Wire format is next.

SEE IT RUN — LIVE DEMO

A spending grant, straight from chat.



1

ASK

“create a spending grant for whiskey, under \$150”

2

APPROVE — INLINE

the consent card renders right in the chat — **\$150 cap, Approve / Decline**

3

VERIFY & SIGN

age-checked, then **device-signed over the exact bounds** — to Utopia · ≤ \$150 · oak-whiskey

4

AUTHORIZED

the agent may now spend within the limits — **hands-free.**

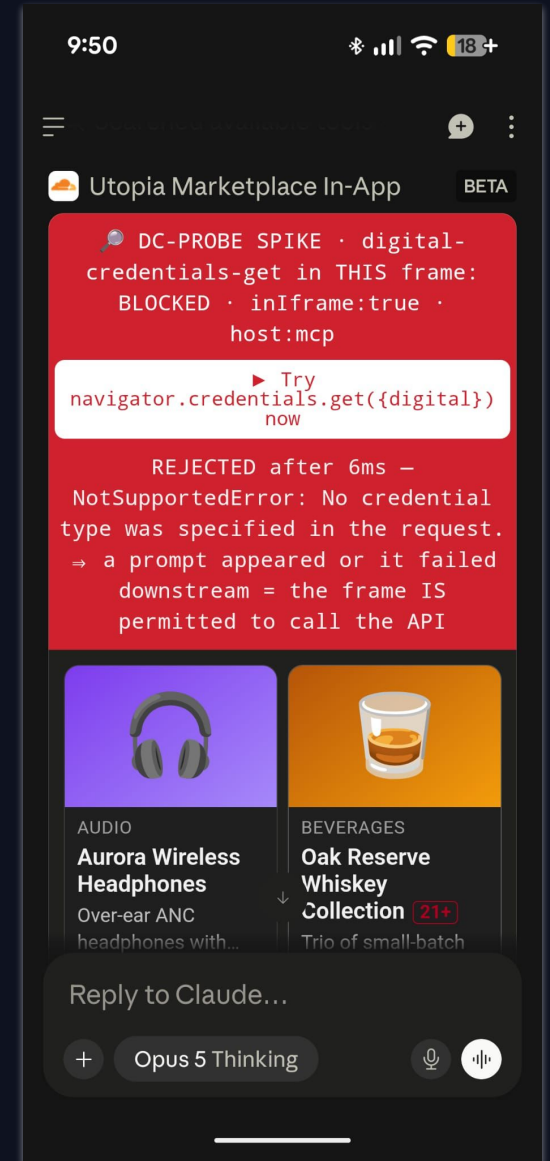
Limits enforced server-side · the device signature is real



Run the consent ceremony inside the agent — no browser handoff.

TODAY Agent → browser tab → wallet → back
NEXT Agent widget → **wallet prompt, inline**

Blocked today by one missing host permission on the widget frame —
allow="digital-credentials-get". A host-platform setting, not new cryptography



WHERE THIS GOES



THE ROADMAP

01 PROTOCOL ROUND TRIP

Server returns `input_required`; client re-issues. No poll, no held-open stream.

polling → **MRTR (SEP-2322)**

02 HUMAN NOT PRESENT

Human pre-signs an Intent Mandate; the agent acts alone within its caps.

dev-signed → **AP2-shaped, Multipaz-SDK-signed**

NOT ON US · WAITING ON THE HOSTS

PAGE-LESS GATING

Unlocks the moment an agent host allows the credential prompt on the widget frame. A platform setting — no new cryptography from us.

`allow="digital-credentials-get"`

IN PARALLEL · NO LOCK-IN

PLUGGABLE, END TO END

Bound to the credential layer — not to one commerce protocol, and not to one chain.

cart protocols

ACP

UCP

MCP

x402 · partial today

Hedera

→

more chains, as plugins



When do we get started?

Paste it into Claude, ChatGPT, Goose, or Gemini — ask it to sell you whiskey. **The age gate fires.**

TRY THE LIVE MCP CONNECTOR

```
https://credentagent-demo-  
dev.vercel.app/mcp
```

ASSEMBLED FROM OPEN STANDARDS

MCP & MCP Apps • ISO 18013-5/7 mdoc • SD-JWT VC • OpenID4VP • DCQL • W3C DC API •
WebAuthn / FIDO2



SLIDES



SCAN → THE REPO

**Try it. Break it. Then
come tell me how — I'll
be around all day.**

Addendum



Authentication is solved. Proof about the person is not.

Agentic commerce is already real — MCP · ACP/UCP · AP2 · x402 · **165M agent transactions** already settled.

✓ SOLVED

OAuth 2.1 — the session is authenticated

AP2 mandate — the buyer approved this cart

*A bearer token can be **stolen and replayed**.*



✗ STILL MISSING

An issuer-signed attribute about the person

Bound to a key that cannot leave the device

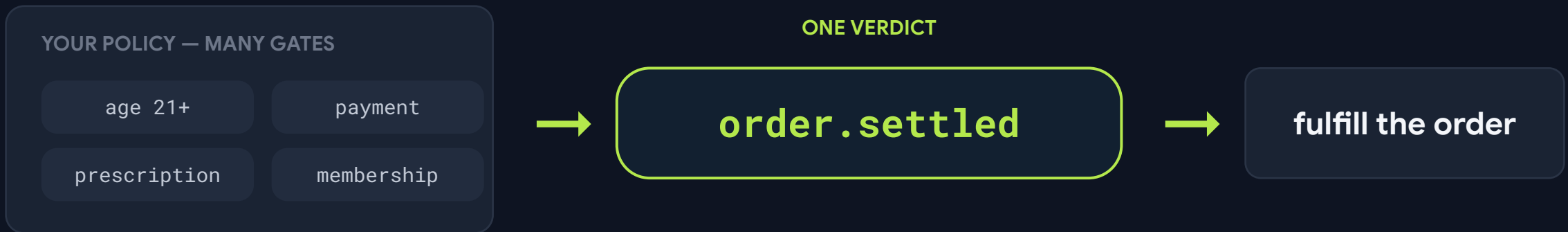
*A key in a secure element **cannot**.*

► **That is why the wallet has to be in the picture at all.**

FOR THE MERCHANT

You don't check gates — you watch one signal

The gate enforces the whole policy inside, fail-closed. Every credential passed and the order settles — or it doesn't. **There is no partial.**



Anything unmet → no `order.settled`. The shopper is sent back to prove the missing credential, then it settles.

HOW YOU RECEIVE IT

Inline you run the host

```
if (result.completed)
```

Webhook recommended

```
on "order.settled"
```

Poll ask anytime

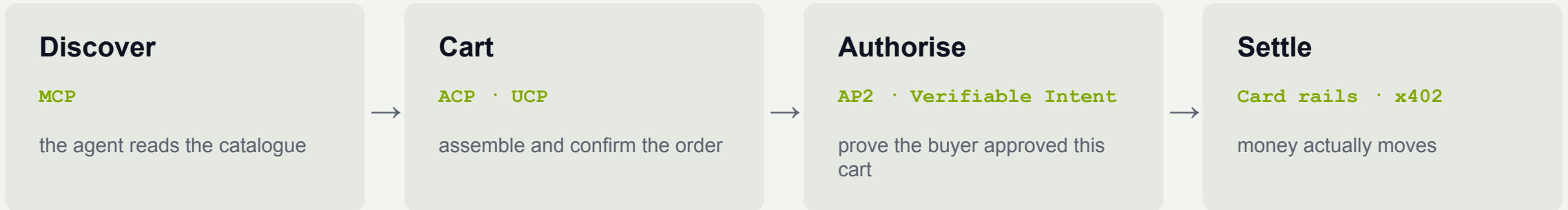
```
GET /orders/:id
```

Same verdict whether you run `createStorefront()` or your own host · github.com/openmobilehub/credentagent



The agentic commerce stack is already real

Identity leads. Payments is one application.



Google · OpenAI · Stripe · Visa · Mastercard · Coinbase · Cloudflare · AWS · Adyen — and x402 alone has moved 165 million agent transactions.

THE GAP

Authentication is solved. Attributes are not.

WHAT THE STACK HAS

- OAuth 2.1 — the session is authenticated
- AP2 mandate — the buyer approved this cart

A bearer token can be stolen and replayed.

WHAT IT STILL LACKS

- An issuer-signed attribute about the person
- Bound to a key that cannot leave the device

A key in a secure element cannot.

That is why the wallet has to be in the picture at all.



Same policy shape, three kinds of action

Payment

```
payment.in("usd")
```

Settles last, after the credential check

Age gate

```
age.over(21)
```

Explicit positive claim, never token-presence

Access grant

```
membership / custom
```

Unlock, entitle, admit — not a purchase

The widget, and where each part runs

Tool mints

The MCP tool returns a checkout link plus a requires manifest

→ Page runs

The wallet ceremony happens in the browser, origin-bound

→ Poll reports

The widget shows the confirmation back inside the agent

TODAY



Multipaz, and the people behind it



Multipaz

Kotlin Multiplatform wallet stack donated by Google to the OpenWallet Foundation. One codebase for Android and iOS, with no proprietary runtime dependency.



Open Mobile Hub

Linux Foundation. CredentAgent is Apache-2.0, published on npm.



Built in the open

Built across two foundations — Open Mobile Hub at Linux Foundation, Multipaz at OpenWallet.



What binds cryptographically — and what does not

Real today

- WebAuthn — origin-bound, replay-protected
- OpenID4VP — Encrypted responses, decrypted and parsed server-side
- ISO mdoc parsing, cross-device via PXP / caBLE
- Enforcement server-side on the completion path

Not yet

- No issuer or device-signature trust anchor
- A self-crafted mdoc would pass today
- AP2-shaped mandate is dev-signed, not key-signed
- Issuer-verified mdoc via Multipaz is the next line
- Agent round trip is poll-based; MRTR is roadmap

`trust_level: "presence-only-demo"` — a flow demo, not a safety control.

main   0  0  Ln 4, Col 20 Spaces: 5 UTF-8 LF { } Markdown

1:



Three seams you actually change



Bring your own host

```
const credentagent = new CredentAgent({  
  walletOrigin: "https://shop.example",  
});  
credentagent.mount(myExpressApp);
```



Conditional policy

```
required(  
  age.over(21).when((order) =>  
    order.lines.some((l) => l.minimumAge !== null)  
  )  
)
```



Optional credentials

```
optional(membership.discount(10))  
// present → 10% off  
// absent → checkout still completes
```

Beyond the built-in builders, `defineCredential` and `dcql` let you describe any credential type you need.